



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SALES DATA ANALYZER

An Automated Python-Based System for Validation, Cleaning,
Statistical Analysis and Visualization of Sales Data

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of
PYTHON PROGRAMMING (CSA 0805)

to the award of the degree of
BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (BRANCH)

Submitted by

J.RAKSAKANTH-1911260099

Under the Supervision of

Guide Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Sales Data Analyzer: An Automated Python-Based System for Validation, Cleaning, Statistical Analysis and Visualization of Sales Data" has been carried out by J.RAKSAKANTH-1911260099 under the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech / B.E. programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Anushya. S

Program Director

Department of CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.C.ROHITH BHAT

PROFESSOR

DEPARTEMENT OF CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I J.RAKSAKANTH-1911260099 , of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Sales Data Analyzer: An Automated Python-Based System for Validation, Cleaning, Statistical Analysis and Visualization of Sales Data" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:03 SEP 2026

Signature of the Students with Names

J.RAKSAKANTH

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
SANJAY GOWTHAM - 1921260091,	Requirement analysis; FileHandler & CSV ingestion module	Designed FileHandler class and header validation logic	Unit-tested CSV import with malformed/valid files	Chapter 1, Chapter 3 (3.1– 3.3)	20%
A.MOHAMED ANIF- 1920260007	Error & exception management module	Designed ErrorManager class (FileNotFoundException, KeyError, ValueError handling)	Tested error logging and malformed-row handling	Chapter 3 (3.6– 3.7)	20%
C.SIBI VELAM - 1920260026,	Data cleaning & transformation module	Designed DataTransformer class (clean_data, filter_by_category, comprehensions)	Validated cleaned dataset against raw dataset	Chapter 4 (4.1– 4.2)	20%
J.RAKSAKANTH- 1911260099	Statistical & advanced analytics engine	Designed StatsEngine and AdvancedAnalyticsEngine (momentum, quartiles, health score)	Verified statistical outputs against manual calculations	Chapter 4 (4.3– 4.6)	20%
S.TAMIL SELVAN - 1911260029	Visualization, dashboard integration & documentation	Designed Visualizer class and Matplotlib dashboard; UML diagram	Validated dashboard outputs and end-to- end pipeline	Chapter 2, Chapter 5, References	20%

Total: 100%

ABSTRACT

Sales data is commonly stored and exchanged in CSV format, but raw datasets frequently contain missing, incomplete, or inconsistent records that must be validated and cleaned before they can yield meaningful business insight. Manual CSV analysis is time-consuming, error-prone, and typically limited to computing basic totals and averages, leaving hidden trends and dataset reliability unmeasured. This project addresses that engineering problem through the design and implementation of the Sales Data Analyzer, a modular, object-oriented Python system that automates the complete journey from raw sales records to actionable visual insight.

The need for the project arises from the limitations of conventional manual workflows: every step from inspection to charting requires human intervention, manual cleaning introduces inconsistencies, and no end-to-end automated pipeline exists that also reports on data quality. The proposed solution is a seven-module architecture — `AppController`, `FileHandler`, `ErrorManager`, `DataTransformer`, `StatsEngine`, `AdvancedAnalyticsEngine`, and `Visualizer` — in which each class has a single, well-defined responsibility and the `AppController` orchestrates the full pipeline end to end.

The methodology follows a structured pipeline: the `FileHandler` imports and validates CSV headers (Date, Category, Amount); the `ErrorManager` logs and gracefully skips malformed rows without halting execution; the `DataTransformer` cleans and type-converts the surviving records; the `StatsEngine` computes total, average, variance, standard deviation, and IQR-based outliers; the `AdvancedAnalyticsEngine` layers on trend momentum, a percentile/quartile profile, and a 0–100 Data Health Score with a quality tier; and the `Visualizer` renders a 2×2 Matplotlib dashboard covering the cumulative sales trend, category-wise sales, transaction values, and the health score.

The major results demonstrate that the system successfully ingests representative sales CSV files, isolates and logs invalid rows without interrupting processing, and produces statistically consistent basic and advanced metrics alongside an automatically generated analytical dashboard image. The principal conclusion is that combining defensive file handling, systematic error management, and layered statistical analytics within a modular object-oriented design in Python provides a reliable, maintainable, and extensible foundation for transforming raw sales data into trustworthy business insight, and lays the groundwork for future extensions such as database persistence, a web interface, and machine-learning-based forecasting.

Keywords

Sales Data Analysis, Data Validation and Cleaning, Python, Object-Oriented Programming, Statistical Analytics, Data Visualization

TABLE OF CONTENTS

CERTIFICATE FROM THE SUPERVISOR.....	ii
DECLARATION BY THE CANDIDATES	iii
INDIVIDUAL CONTRIBUTION STATEMENT	iv
ABSTRACT.....	v
LIST OF FIGURES.....	vi
LIST OF TABLES.....	vii
LIST OF ABBREVIATIONS	viii
CHAPTER 1: Introduction and Engineering Problem.....	1
CHAPTER 2: Literature Review and Existing Solutions.....	4
CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security.....	6
CHAPTER 4: Implementation, Testing & Results, Project Management.....	10
CHAPTER 5: Conclusion, Future Work and Reflection	14
REFERENCES	16
APPENDICES	17

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Processing Pipeline (CSV Input → Dashboard)	2
3.1	Proposed System Architecture	7
3.2	System Modules Overview	8
4.1	System Workflow Flowchart	12
4.2	UML Class Diagram	13
4.3	Analytical Dashboard (2×2 Matplotlib layout)	13

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis of Existing Approaches	5
3.1	Stakeholder / User Requirements	6
3.2	Functional & Non-Functional Requirements	6
3.3	Design Specifications	7
3.4	Alternative Solutions Evaluation Matrix	8
3.5	Trade-off Analysis	9
3.6	Sustainability, Ethics, Safety, Security	9
3.7	Risk Register	10
4.1	Tools / Software Used	10
4.2	Test Plan & Verification	11
4.3	Achieved vs Required Values	11
4.4	Achievement of Objectives	12
4.5	Roles, Responsibilities & Contribution	13
4.6	Project Cost Table	13

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CSV	Comma-Separated Values	—
OOP	Object-Oriented Programming	—
IQR	Interquartile Range	—
Q1 / Q3	First Quartile / Third Quartile	—
CV	Coefficient of Variation	—
UI	User Interface	—
ML	Machine Learning	—
API	Application Programming Interface	—
UML	Unified Modeling Language	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Sales data is commonly stored in CSV files, but raw datasets require validation, cleaning and statistical processing before meaningful insights can be obtained. Organisations routinely export transactional sales records into flat CSV files from point-of-sale systems, e-commerce platforms, or spreadsheets, and analysts are then expected to import, verify, and analyse this data before it can support decision-making. Python, with its strong ecosystem of file-handling, data-processing and visualization libraries, provides a practical technical foundation for automating this pipeline through an object-oriented, modular architecture.

1.2 Need for the Project

Challenge	Description
Manual Effort	Manual CSV analysis is time-consuming.
Invalid Records	Raw datasets may contain incomplete or invalid records.
Basic Stats Only	Basic programs often provide only totals and averages.
Error-Prone Cleaning	Manual cleaning can introduce human errors.
Hidden Trends	Sales trends may not be clearly identified.
No Health Metrics	Dataset reliability is often not measured.

1.3 Problem Statement

How can raw sales data be automatically validated, cleaned, statistically analysed, evaluated for quality, and transformed into meaningful visual insights? The problem is engineering-significant because it involves multiple interacting factors — file-format validation, defensive error handling, numerical statistics, and visualization — under the realistic constraint that any single malformed record must not be allowed to halt the entire analysis pipeline.

1.4 Project Aim

To design and implement an automated, modular, object-oriented Python system — the Sales Data Analyzer — that ingests raw CSV sales data, validates and cleans it, computes basic and advanced statistical analytics, evaluates dataset quality, and generates an automated visual dashboard.

1.5 Project Objectives

No.	Objective	Description
1	Robust Data Ingestion	CSV import and header validation using FileHandler.

No.	Objective	Description
2	Data Cleaning	Identify and handle invalid records via DataTransformer.
3	Statistical Analysis	Compute total, average, variance, standard deviation and outliers.
4	Advanced Analytics	Compute Trend Momentum, Quartile Profiling and a Health Score Index.
5	Visualization	Generate an automated analytical dashboard using Matplotlib.

1.6 Scope

Included: CSV-based sales datasets with Date, Category and Amount fields; automated validation, cleaning, basic and advanced statistical analytics; automated dashboard generation.

Excluded: real-time streaming ingestion, database persistence, and a graphical/web user interface (identified as future scope).

Assumptions: input files are UTF-8 encoded CSV files; the Amount field represents a non-negative numeric transaction value.

Boundary conditions: the system processes one CSV file per execution and produces one dashboard image per run.

1.7 Expected Engineering Outcome

The expected outcome is a working software system (a Python application/prototype) comprising the FileHandler, ErrorManager, DataTransformer, StatsEngine, AdvancedAnalyticsEngine and Visualizer classes, orchestrated by an AppController, that outputs a cleaned dataset, a statistical summary, and an analytical dashboard image (sales_dashboard.png).

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant existing approaches were identified by examining conventional manual sales-analysis workflows and common spreadsheet-based practices used in small and medium business settings, alongside standard Python data-handling and visualization libraries (csv, statistics, Matplotlib) referenced in the project's technical documentation.

2.2 Review of Existing Work

The conventional manual workflow for sales-data analysis follows a linear sequence: CSV → Manual Inspection → Manual Cleaning → Spreadsheet Calculations → Manual Charts. Every step in this workflow requires direct human intervention, manual cleaning is prone to introducing inconsistencies, and there is no end-to-end automated pipeline linking ingestion to visualization. In addition, the statistics such workflows typically produce are basic — totals and averages — with no formal error-management step and no measurement of dataset reliability, so invalid rows often go unnoticed and hidden trends are not systematically surfaced.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual spreadsheet analysis	Low tooling overhead; familiar to most users	Time-consuming, error-prone, basic statistics only	Motivates automation of ingestion, cleaning and statistics
Ad-hoc scripted totals/averages	Faster than fully manual work	No structured error handling; no data-quality scoring	Motivates ErrorManager and Data Health Score design
Generic BI/dashboard tools	Rich visualization capability	Often assume already-clean data; limited transparency into cleaning logic	Motivates a transparent, code-level cleaning and analytics pipeline

2.4 Research / Engineering Gap

Existing manual and semi-manual approaches leave three gaps unresolved: (i) no automated, resilient validation-and-cleaning stage that gracefully skips malformed rows while logging them; (ii) no layered analytics that go beyond totals and averages to trend momentum and quartile-based profiling; and (iii) no explicit, quantified measure of dataset reliability such as a data health score.

2.5 Proposed Contribution

The Sales Data Analyzer addresses these gaps through a modular, object-oriented pipeline in which a dedicated `ErrorManager` provides resilience against file and data errors without halting execution, an `AdvancedAnalyticsEngine` (inheriting from `StatsEngine`) adds trend momentum, percentile profiling and a 0–100 Data Health Score with a quality tier, and a `Visualizer` automatically renders a four-panel analytical dashboard — combining automation, statistical depth, and data-quality transparency in a single maintainable system.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Sales analyst / end user	Import a CSV file and receive validated, cleaned data automatically.
Business decision-maker	Obtain trustworthy statistics and a visual dashboard summarising sales performance and data quality.
Developer / maintainer	A modular, well-documented codebase that is easy to extend (e.g., new metrics, new data sources).

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Validate presence of required CSV headers (Date, Category, Amount)	100% of required headers checked before processing
Functional	Skip and log malformed rows instead of terminating	0 unhandled exceptions on malformed input rows
Functional	Compute basic statistics (total, average, variance, std. dev., outliers)	All five metrics computed per run
Functional	Compute advanced analytics (momentum, percentile profile, health score)	Health score reported on a 0–100 scale with a tier label
Non-Functional	Generate a dashboard image automatically	One PNG dashboard produced per successful run
Non-Functional	Maintainability	Each responsibility isolated in its own class/module

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
PEP 8 (Python Style Guide)	Consistent, readable Python code layout and naming	Applied across FileHandler, ErrorManager, DataTransformer, StatsEngine, Visualizer classes
RFC 4180 (CSV Format)	Well-formed comma-separated value files	FileHandler uses csv.DictReader with utf-8-sig encoding to read CSV files consistent with the convention

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Required CSV headers	Date, Category, Amount	—	High
Outlier detection method	Interquartile Range (IQR)	—	High
Health Score range	0–100	score	Medium
Dashboard layout	2×2 subplot grid	—	Medium
Dashboard resolution	150	dpi	Low

3.4 Alternative Solutions & Evaluation

Alternative 1 (Manual spreadsheet workflow): Analysts import CSV data into a spreadsheet and compute totals/averages and charts by hand.

Alternative 2 (Generic BI dashboard tool): Sales data is fed into a third-party business-intelligence tool for visualization, assuming the data is already clean.

Alternative 3 (Proposed automated Python pipeline): A custom object-oriented pipeline performs validation, cleaning, basic and advanced statistics, and dashboard generation end to end.

Criterion	Weight	Alt. 1 — Manual	Alt. 2 — BI Tool	Alt. 3 — Proposed
Performance	20%	Low	Medium	High
Cost	20%	Low tooling cost, high labour cost	Licensing cost	Low (open-source Python stack)
Safety (data integrity)	20%	Low	Medium	High
Sustainability (reuse/extension)	20%	Low	Medium	High
Reliability (error handling)	20%	Low	Assumes clean input	High

3.5 Selected Design & Detailed Design

Alternative 3 was selected because, unlike Alternative 1, it removes manual effort and human error from validation, cleaning and computation, and unlike Alternative 2, it does not assume the input data is already clean — it performs its own defensive validation and reports data-quality metrics, evidenced by the systematic comparison in Table 3.4.

Proposed System Architecture

CSV Input (sales_data.csv) → AppController, which orchestrates the complete pipeline through six collaborating classes:

- FileHandler — Load & Validate Headers (Modular: clear separation of concerns)
- ErrorManager — Log & Skip Malformed Rows (Automated: end-to-end processing)
- DataTransformer — Clean & Transform Records (Error-aware: graceful failure handling)

- StatsEngine — Basic Statistics & Outliers (Statistically rich: basic + advanced metrics)
- AdvancedAnalyticsEngine — Momentum · Quartiles · Health (Visual: integrated dashboard generation)
- Visualizer — Matplotlib Dashboard (Maintainable: OOP architecture)

→ Analytical Dashboard (sales_dashboard.png)

System Modules

Module	Responsibility	File
M1	File Handling & CSV Import	file_handler.py
M2	Exception & Error Management	error_handler.py
M3	Data Transformation & Cleaning	data_controller.py
M4	Statistical Aggregation & Advanced Analytics	stats_engine.py
M5	Visualization & Dashboard	visualizer.py

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Skip malformed rows vs. halt on first error	Terminate processing on the first invalid row	Simpler control flow	One bad row would block analysis of an otherwise valid dataset	Skip and log via ErrorManager — maximises usable data while preserving a full audit trail
IQR-based outlier detection vs. z-score	Z-score method	Familiar, simple to compute	Sensitive to non-normal distributions common in sales data	IQR method chosen for robustness to skewed sales distributions
In-memory processing vs. database-backed pipeline	Persist records in a database from the start	Durable, queryable storage	Added infrastructure complexity for the current project scope	In-memory processing chosen for this phase; database integration deferred to future work (Section 5.2)

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reusability and extensibility of the codebase	Modular class design was compared against a monolithic script approach	Adopted a modular OOP architecture so future analytics or data sources can be added without rewriting existing modules

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Ethics	Integrity of reported statistics and dashboards	Reviewed whether cleaning logic could silently distort results	Malformed rows are logged and reported (not silently discarded) so data-quality is transparent to end users
Health & Safety	Not a physical/hardware system — no direct occupational hazard	Reviewed system scope; confirmed purely software-based operation	No physical safety mitigations required; noted explicitly rather than assumed
Security	Protection against malformed or malicious input files	Reviewed FileNotFoundError, KeyError, ValueError handling in ErrorManager	Defensive exception handling implemented around file access and record parsing to prevent uncontrolled crashes

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Missing/incorrect CSV headers	Medium	Medium	Medium	verify_headers() checks REQUIRED_HEADERS before processing and raises a clear error	Low
Malformed or missing field values	High	Low	Medium	DataTransformer validates each field and ErrorManager logs/skips the row	Low
Negative or invalid Amount values	Low	Medium	Low	Explicit ValueError raised for negative amounts	Low
Input file not found	Low	Medium	Low	handle_file_not_found() in ErrorManager reports the error cleanly	Low

UML Class Diagram (summary)

Class	Key Attributes	Key Methods
AppController	file_path	run_pipeline(); display_summary()
FileHandler	filename, raw_data	load_csv(); verify_headers()
ErrorManager	error_log, skipped_rows	log_error(); handle_file_not_found(); handle_malformed_row()
DataTransformer	raw_rows, cleaned_rows	clean_data(); filter_by_category(); apply_comprehensions()
StatsEngine	data	compute_basic_statistics(); detect_outliers()
AdvancedAnalyticsEngine (inherits StatsEngine)	summary_cache	compute_advanced_metrics(); compute_cumulative_and_velocity(); compute_percentile_profile(); generate_data_health_index()

Class	Key Attributes	Key Methods
Visualizer	stats_data, fig, ax	generate_analytical_dashboard(); plot_category_distribution()

Python & OOP Concepts Applied

OOP Concept	Application in This Project
Encapsulation	Data and methods combined inside each class (e.g., FileHandler, ErrorManager)
Inheritance	AdvancedAnalyticsEngine(StatsEngine)
Abstraction	Public interfaces (e.g., run_pipeline()) hide internal processing details
Objects / Classes	Modular, reusable components composed by AppController

Python features used: lists & dictionaries, list comprehensions, exception handling, file & CSV handling, and Matplotlib visualization.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an incremental, module-by-module approach in Python: each of the five modules (file handling, error management, data transformation, statistical/advanced analytics, visualization) was implemented and unit-verified in isolation before being integrated under the AppController, following object-oriented principles (encapsulation, inheritance, abstraction) throughout.

Resource	Purpose
Python 3	Core implementation language
csv module	Reading and parsing CSV sales data
statistics / math	Basic statistical computation
Matplotlib	Dashboard generation
Sample sales_data.csv dataset	Input dataset for testing and demonstration

4.2 Software Implementation & Modern Tools Used

Module 1 — File Handling & CSV Import (file_handler.py, class FileHandler)

Responsible for safely loading the CSV dataset and verifying its required structure (Date, Category, Amount). REQUIRED_HEADERS = ["Date", "Category", "Amount"]. load_csv() opens the file with utf-8-sig encoding and reads it with csv.DictReader; verify_headers() checks the parsed field names against REQUIRED_HEADERS and raises a KeyError if any are missing, before the raw records are returned.

Module 2 — Error & Exception Management (error_handler.py, class ErrorManager)

Provides resilience against file and data-related errors, handling FileNotFoundError, KeyError, ValueError, and malformed records. handle_malformed_row() increments a skipped-row counter and logs the row; handle_file_not_found() reports a missing input file; log_error() appends a formatted entry (with row number where applicable) to an internal error log for later review.

Module 3 — Data Transformation & Cleaning (data_controller.py, class DataTransformer)

Converts raw CSV records into clean, usable analytical data via clean_data(), filter_by_category(), and apply_comprehensions(). Each raw row's Date, Category and Amount fields are stripped and validated; missing fields or a non-numeric/negative Amount raise a ValueError, which is routed to the ErrorManager rather than terminating the run, and only successfully validated rows are appended to the cleaned dataset.

Module 4 — Statistical Aggregation & Advanced Analytics (stats_engine.py, classes StatsEngine, AdvancedAnalyticsEngine)

Basic statistics (compute_basic_statistics(), detect_outliers()):

- Total, Average, Variance, Standard Deviation, Minimum/Maximum, Range
- Outlier detection using the IQR method: values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are flagged

Advanced analytics (AdvancedAnalyticsEngine, which inherits StatsEngine):

- Trend Momentum — cumulative running total, with Momentum (%) = $((\text{Current} - \text{Previous}) / \text{Previous}) \times 100$
- Percentile Profile — Minimum, Q1, Median, Q3, Maximum via a quartile/percentile profiler
- Data Health Score — a 0–100 quality-confidence index combining retention rate, coefficient of variation and a volatility penalty, mapped to a tier: Production / Good / Moderate / Needs Review

Module 5 — Visualization & Dashboard (visualizer.py, class Visualizer)

Converts analytical results into a 2x2 Matplotlib dashboard: Cumulative Sales Trend, Sales by Category, Transaction Values, and Health Score. The figure is built with `plt.subplots(2, 2, figsize=(12, 8))`; `axes[0,0]` plots the cumulative trend, `axes[0,1]` bar-charts sales by category, `axes[1,0]` plots individual transaction values, and `axes[1,1]` bar-charts the health score against a fixed 0–100 y-axis. The figure is saved as `sales_dashboard.png` at 150 dpi with tight bounding-box cropping.

Tool / Software	Purpose	Stage Used
Python 3 / csv module	CSV parsing and file I/O	Module 1
Custom exception handling (try/except)	Defensive error management	Modules 2–3
List comprehensions, statistics/math	Data cleaning and statistical computation	Modules 3–4
Matplotlib	Dashboard rendering	Module 5

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Header validation	Load CSV files with missing/renamed headers	KeyError raised and reported; processing halts safely before analysis
Malformed row handling	Load CSV files containing blank fields and non-numeric Amount values	Row is skipped, logged, and processing continues
Negative amount rejection	Load rows with a negative Amount value	ValueError raised for that row; row excluded from cleaned dataset

Requirement	Test Method	Acceptance Criterion
Outlier detection	Run <code>detect_outliers()</code> against a dataset with known extreme values	Values outside the IQR fence are correctly flagged
Dashboard generation	Run the full pipeline on a valid dataset	<code>sales_dashboard.png</code> generated with all four panels populated

Specification	Required Value	Achieved Value	Status
Required headers validated	Date, Category, Amount	Date, Category, Amount	Pass
Outlier detection method	IQR method	IQR method implemented in <code>detect_outliers()</code>	Pass
Health Score range	0–100	0–100 (with tier label)	Pass
Dashboard panels	4 (2×2 layout)	4 (2×2 layout)	Pass

4.4 Results

Running the pipeline on a representative `sales_data.csv` produces a cleaned dataset (with malformed rows logged and excluded), a basic statistics summary (total, average, variance, standard deviation, min/max, range, outliers), advanced analytics (cumulative trend momentum, percentile profile, and Data Health Score with tier), and the four-panel analytical dashboard shown below.

- Panel 1 — Cumulative Sales Trend (line plot with markers over transaction dates)
- Panel 2 — Sales by Category (bar chart)
- Panel 3 — Transaction Values (line plot of individual amounts)
- Panel 4 — Health Score (bar chart, fixed 0–100 scale)

4.5 Data Analysis & Engineering Judgment

The IQR-based outlier detection consistently isolates transactions that deviate sharply from the typical sales range without being distorted by dataset skew, which is preferable to a z-score approach for the non-normal distributions typical of sales data. Where the retention rate (proportion of rows surviving cleaning) was lower than expected during testing with intentionally corrupted sample files, the Data Health Score and its tier label correctly reflected the reduced data quality, confirming that the health-scoring logic responds appropriately to real data-quality issues rather than masking them.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Robust Data Ingestion	<code>FileHandler.load_csv()</code> and <code>verify_headers()</code> implemented and tested against valid/invalid header sets	Fully Achieved

Objective	Evidence	Achievement
Data Cleaning	DataTransformer.clean_data() validates and type-converts records; invalid rows routed to ErrorManager	Fully Achieved
Statistical Analysis	StatsEngine.compute_basic_statistics() and detect_outliers() implemented and verified	Fully Achieved
Advanced Analytics	AdvancedAnalyticsEngine implements momentum, percentile profile and health score	Fully Achieved
Visualization	Visualizer generates a 2x2 Matplotlib dashboard saved as sales_dashboard.png	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Automated — reduces manual processing end-to-end
- Modular — separate responsibilities per module (AppController, FileHandler, ErrorManager, DataTransformer, StatsEngine, AdvancedAnalyticsEngine, Visualizer)
- Robust — handles common file and data errors gracefully via ErrorManager
- Analytical — provides both basic and advanced statistics
- Innovative — three advanced Module-4 features (trend momentum, percentile profiling, health score)
- Visual — automated Matplotlib dashboard generation
- Maintainable — clean OOP architecture with inheritance (AdvancedAnalyticsEngine extends StatsEngine)

Limitations:

- Processes one CSV file per run; no batch or streaming ingestion
- No persistent storage — results are not saved to a database between runs
- No graphical or web-based user interface; currently a script-driven pipeline
- Statistical and health-score thresholds are currently fixed rather than user-configurable

4.8 Project Planning, Roles & Budget

Milestone Timeline

Milestone	Planned Duration	Status
Requirement analysis & design	Week 1–2	Completed
Module 1–2 implementation (FileHandler, ErrorManager)	Week 3–4	Completed
Module 3 implementation (DataTransformer)	Week 5	Completed
Module 4 implementation (StatsEngine, AdvancedAnalyticsEngine)	Week 6–7	Completed

Milestone	Planned Duration	Status
Module 5 implementation (Visualizer) & integration	Week 8	Completed
Testing, documentation & report writing	Week 9–10	Completed

Roles & Responsibilities

Student	Assigned Responsibility	Actual Contribution
Student 1	FileHandler & CSV ingestion module	Implemented and tested Module 1
Student 2	ErrorManager module	Implemented and tested Module 2
Student 3	DataTransformer module	Implemented and tested Module 3
Student 4	StatsEngine & AdvancedAnalyticsEngine	Implemented and tested Module 4
Student 5	Visualizer, dashboard & documentation	Implemented Module 5 and coordinated report writing

Project Cost Table

Item	Estimated Cost	Actual Cost
Development tools (Python, open-source libraries)	₹0 (open-source)	₹0 (open-source)
Computing resources (personal laptops)	₹0 (existing hardware)	₹0 (existing hardware)
Miscellaneous (printing, documentation)	₹500	₹500

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Sales Data Analyzer provides a complete automated pipeline for transforming raw CSV sales records into meaningful analytical information. The system successfully implements CSV import, header validation, resilient error handling, data cleaning, basic statistics, outlier detection, trend momentum, quartile analysis, a data health score, automated visualization, and a maintainable object-oriented architecture. The project demonstrates how Python and Object-Oriented Programming can transform raw sales data into reliable statistical, analytical and visual business insights through a modular and extensible system.

5.2 Future Work

No.	Future Direction	Description
1	Database Integration	Persistent storage via PostgreSQL or MongoDB, replacing in-memory processing
2	Web Interface	An interactive UI (Flask / Streamlit) with real-time CSV upload
3	Machine Learning	Sales forecasting, predictive analytics and trend prediction

Future architecture: Database / API → Processing Layer → Analytics Engine → Machine Learning → Web Dashboard.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of Python object-oriented design (encapsulation, inheritance, abstraction) to a real data-processing pipeline
- Defensive programming and structured exception handling for file- and data-related errors
- Statistical methods including IQR-based outlier detection, percentile profiling and composite quality scoring
- Automated dashboard generation using Matplotlib

Engineering & Professional Skills Developed:

- Modular system design and separation of concerns across collaborating classes
- Requirement analysis, trade-off evaluation, and structured technical documentation
- Collaborative, module-based teamwork with individually verifiable contributions

Individual Reflection (each student, 4–5 lines):

- Most difficult engineering problem encountered and how it was solved — [to be completed by each student]

- A key engineering decision made personally and the evidence behind it — [to be completed by each student]
- New knowledge acquired independently — [to be completed by each student]
- What would be redesigned if repeated — [to be completed by each student]

REFERENCES

- [1] Python Software Foundation, "csv — CSV File Reading and Writing," Python 3 Documentation.
- [2] Python Software Foundation, "statistics — Mathematical statistics functions," Python 3 Documentation.
- [3] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, 2007.
- [4] Python Enhancement Proposal 8 (PEP 8) — Style Guide for Python Code.
- [5] RFC 4180 — Common Format and MIME Type for Comma-Separated Values (CSV) Files.
- [6] Sales Data Analyzer — Academic Project Presentation (project source materials), 2026.

APPENDICES

Appendix A – Detailed Calculations

Momentum (%) = $((\text{Current} - \text{Previous}) / \text{Previous}) \times 100$

IQR fence: Lower = $Q1 - 1.5 \times \text{IQR}$; Upper = $Q3 + 1.5 \times \text{IQR}$, where $\text{IQR} = Q3 - Q1$

Appendix B – System Workflow (Flowchart)

START → Load sales_data.csv → Validate Headers → Read & Validate Each Row → Valid Row? [NO → Log Error / Skip] [YES → Clean Data] → Calculate Statistics → Advanced Analytics (Trend Momentum → Quartile Analysis → Health Score) → Generate Charts → Dashboard → END

Appendix C – Source Code / Code Repository Information

Core modules: file_handler.py (FileHandler), error_handler.py (ErrorManager), data_controller.py (DataTransformer), stats_engine.py (StatsEngine, AdvancedAnalyticsEngine), visualizer.py (Visualizer). Complete source code is maintained in the project's approved repository; only essential excerpts are included in Chapter 4.

Appendix D – Datasheets

Not applicable — this is a software-only project.

Appendix E – Experimental / Raw Data

Sample input: sales_data.csv (Date, Category, Amount columns). Output: sales_dashboard.png.

Appendix F – Ethics / Institutional Approval

Not applicable — no human-subject or sensitive data was used; only a synthetic/sample sales dataset.

Appendix G – Project Meeting / Progress Records

Team progress was tracked against the milestone timeline in Section 4.8.

Appendix H – User/Industry Feedback

Not applicable for this phase.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at the time of submission.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (page iv) and Section 4.8 (Roles & Responsibilities).

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department/faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)